PROCESAMIENTO DEL LENGUAJE NATURAL

Práctica 1: Preparación y procesamiento de textos

Universidad Autónoma de Madrid

Iván Domínguez Hernández | ivan.dominguezh@estudiante.uam.es

Lucas Miranda López | lucas.mirandal@estudiante.uam.es

Índice

1. Descarga y preparación de la información **2**

2. Diseñar e implementar el gestor del corpus **3**

3. Preprocesamiento del corpus y análisis básicos **4**

4. Comentarios y conclusiones **4**

4.1. Comentarios 4

4.2. Conclusiones 5

1. Descarga y preparación de la información

Resumen: En esta primera sección se descarga de uno los ficheros del corpus (o base de datos textuales) de comentarios de *Amazon* (uno sencillo y poco pesado, el de tarjetas de regalo) y se diseña la clase `ProcesadorDeTexto` para almacenar cada review en un fichero propio en el directorio que le corresponda sin almacenar reviews duplciadas.

1. La decisión de descargar el fichero *Gift_ Cards.jsonl* se debe únicamente a su tamaño. Teniendo en cuenta que todos ficheros del corpus de *Amazon* siguen la misma estructura JSON, lo óptimo para la realización de la práctica parecía ser escoger un fichero ligero sobre el que hacer pruebas en un archivo dde tipo *.ipynb* para, posteriormente, codificar sencillamente el archivo *.py* definitvo y genérico capaz de procesar cualquier fichero del corpus.

2. Las pruebas realizadas sobre un archivo *.ipynb* auxiliar, de prueba, y que no se entrega adjunto, permitieron construir la clase `ProcesadorDeTexto`, capaz de:

   a) Recibir el nombre del fichero a preprocesar (sin especificar su extensión, solo el nombre del fichero). En nuestro caso 'Gift_Cards'.

   b) Recibir una lista con las categorías que el usuario quiera almacenar para cada reseña. Nosotros aconsejamos pasarle la lista `['rating', 'title', 'text', 'parent_asin', 'user_id']`, que almacena la puntación en estrellas del 1 al 5, el título (tal vez contenga información relevante), el texto per se de la reseña, una referencia al tipo de producto reseñado (que posteriormente permitirá acceder a metainformación de ser preciso) y una referencia al usuario que hace la reseña. **Nota:** la característica `'user_id'` siempre es necesaria en la lista para asegurar que se pueden borrar los duplicados, como se explica en el siguiente punto. Una vez usada, se elimina.

   Y ahora, iterando sobre cada elemento del *.jsonl* a procesar:

   c) No almacenar reseñas que, siendo del mismo usuario y al mismo producto, tienen el mismo texto y título. Se puede pensar que esto debería ser parte de un posterior preprocesado. Pero al implicar iterar sobre cada elemento de nuevo, decidimos hacerlo en este paso para ahorrar cómputo a posteriori.

   d) Guardar las categorías enunciadas en la lista que se le pase de cada reseña en un fichero propio, en tres distintos directorios: */positivas, /neutras* y */negativas*, a modo de etiqueta, basándonos únicamente a cual de los intervalos pertenece la categoría *'rating'*: $rating \geq 4.5$, $4.5 \geq rating \geq 1.5$ o $rating \leq 1.5$. Es un criterio absurdo, pero al ser configurable, de momento no nos preocuparemos por esto en esta práctica.

Por tanto, lo que podemos obtener tras esta sección es una división en directorios, basada en la puntuación de cada reseña, en los que se incluyen ficheros *.json*, para cada reseña, y que contienen las categorías especificadas por el usuario. Esto facilitará en gran medida la posterior ingesta (2) de datos por un gestor de corpus especializado en nuestro caso, `AmazonCorpusReader`. **Nota:** los directorios que se crean están bastante desbalanceados. Esto es, existen muchas más reseñas de clase positiva que de clases neutra o negativa. Esto es, en realidad, una distribución inherente, no solo al criterio seguido para el etiquetado, sino también a los propios datos. Y es que lo normal es que los usuarios hagan tantas reseñas negativas o neutras como positivas.

2. Diseñar e implementar el gestor del corpus

Resumen: Para poder acceder al corpus en diferentes niveles (documento, párrafo, oración y palabra como indica el enunciado), así como a la categoría de cada reseña, se desarrolla la clase `AmazonCorpusReader`, heredera de `CategorizedCorpusReader` y `CorpusReader`. La estructura de la clase está fuertemente inspirada en `HTMLCorpusReader` propuesta en el libro [1].

1. **Herencia:** en el constructor (`__init__`) se inicializan ambas clases base por separado:

   a) `CategorizedCorpusReader.__init__` recibe un `cat_pattern`, una expresión regular (`r'(positivas|neutras|negativas)/.*'`) que permite deducir automáticamente la categoría (`categories`) de cada fichero a partir del directorio en el que se encuentra.

   b) `CorpusReader.__init__` recibe la ruta raíz del corpus y el patrón de los ficheros a considerar (`fileids`, por defecto `r'.*\.json'`).

2. **Método `_resolve()`**: dado que la mayoría de métodos de la clase pueden recibir una lista de `fileids` o una lista de `categories`, este método auxiliar (inspirado en el patrón que sigue `CategorizedPlaintextCorpusReader` en NLTK) unifica ambos casos: si se reciben categorías, las traduce a la lista de ficheros correspondiente mediante `self.fileids(categories)`; si se reciben ambas a la vez, lanza un `ValueError`, al no tener sentido combinarlas.

3. **Método `docs()`**: recorre uno a uno los ficheros seleccionados (vía `_resolve()`) y, para cada línea del `.json` correspondiente, construye un diccionario con sus campos (`title`, `text`, `rating`, `parent_asin`...) y le añade la clave `'category'` obtenida mediante el método `_resolve_with_category()`. El resto de métodos de acceso hacen uso de él.

4. **Métodos de acceso a campos concretos:** a partir de `docs()` se construyen, como generadores, `titles()`, `ratings()` y `texts()`, que devuelven respectivamente cada uno el título, la puntuación y el texto de cada reseña seleccionada, y `raw()`, que devuelve el contenido en crudo de los ficheros unidos, reutilizando la función `concat()` de NLTK.

5. **Métodos de segmentación:** implementa la jerarquía habitual de un corpus reader de NLTK.

   a) `paras()`: para cada reseña, separa su texto en párrafos (`BlanklineTokenizer`), cada párrafo en oraciones (`sent_tokenize`) y cada oración en palabras (`word_tokenize`), devolviendo una estructura anidada lista → lista → lista.

   b) `sents()`: itera sobre `paras()` y «desenvuelve» un nivel, devolviendo cada oración (como lista de palabras) de forma independiente.

   c) `words()`: itera a su vez sobre `sents()` y devuelve cada palabra individual del corpus seleccionado.

   Todos estos métodos están implementados como generadores (`yield`), de forma que el corpus nunca se carga por completo en memoria, sino que se recorre reseña a reseña.

6. **Método `describe()`:** realiza una única pasada sobre el corpus (o, más bien, el subconjunto de `fileids`/`categories` indicado) y devuelve un diccionario con las métricas habituales de caracterización de corpus: número de ficheros (`files`) y de categorías (`topics`) consideradas; número total de párrafos, oraciones y palabras (`paras`, `sents`, `words`); tamaño del vocabulario (`vocab`); diversidad léxica (`lexdiv`, palabras entre vocabulario); párrafos por documento (`ppdoc`); oraciones por párrafo (`sppar`); y tiempo de cómputo empleado (`secs`). Para ello utiliza dos objetos `nltk.FreqDist()` (uno para los contadores globales y otro para la frecuencia de cada token), evitando recorrer el corpus más de una vez.

3. Preprocesamiento del corpus y análisis básicos

El preprocesamiento se incorpora a nivel de corpus reader. Para hacerlo, fue necesario (dejando de lado la actualización de NLTK), si bien no modificar la funcionalidad ya implementada en la sección 2, añadir bloques lógicos en algunos puntos para la propia implementación de la nueva funcionalidad de preprocesado. En particular, los cambios fueron los siguientes:

1. Creación del método `_limpiar()`: limpia el texto de acuerdo con los requisitos del enunciado de la práctica, en el siguiente orden:

   a) Eliminar etiquetas HTML, de haberlas. Todas pasan a espacio en blanco, salvo `<br />` que pasa a ser salto de línea.

   b) Convierte entidades `&amp` → `&`.

   c) Elimina emojis.

   d) Reduce puntuación repetida !!! → !.

   e) Reduce letras repetidas *goooooood* → *good*.

   f) Normaliza espacios conservando saltos de línea.

   Con tan solo añadir la línea `obj['text'] = self._limpiar(obj['text'])`, ahora todo el resto de métodos (`texts()`, `paras()`, `sents()`, `words()` y `describe()`) reciben el texto ya limpio.

2. El método `paras()` se rehace para evitar blancos: el método ahora comprueba en cada nivel (`para.strip()`, `sent.strip()`, `w.strip()`) y si es blanco se descarta (manteniendo el `yield`).

3. Método `_get_stopwords`: los métodos `paras()`, `sents()` y `words()` reciben ahora un parámetro `sin_stopwords = False`. Con `False`, se eliminan las que NLTK asume como stopwords (en inglés) a excepción de 'no', 'not' y 'nor' para evitar cambios de significado, con `True`, no.

4. Retoques en el método `describe()`: ahora tiene en cuenta el número total de caracteres (se nos olvidó hacerlo) en la sección 2, además de aceptar el parámetro `sin_stepwords`.

4. Comentarios y conclusiones

4.1. Comentarios

1. Tal vez el hecho de eliminar repetidos no fuera una tarea de la sección 1, pues consiste más bien en preprocesamiento. Sin embargo, se nos ocurrió hacerlo el primer día de práctica, sin mucha visión de lo que sería el resultado final de la misma, por lo que decidimos dejarlo en esa sección. Además, se gana en términos de eficiencia, pues ya no es necesario volver a iterar sobre cada uno de los comentarios del corpus.

2. Como propuesta exploratoria para otras prácticas, es probable que convenga integrar la clase `ProcesadorDeTexto` con herramientas de procesamiento por lotes y distribuido, pues es una tarea muy pesada y paralelizable. Un comentario también relacionado con esta clase es que el nombre escogido no es el mejor, pues no es un procesador ni tampoco procesa texto como tal. Su tarea es más bien darle una estructura de corpus a los datos ofrecidos por *Amazon*.

3. No teníamos muy claro si era lo suyo eliminar todos los comentarios escritos en un idioma distinto del inglés. Nuestra teoría es que si no hay mucho desbalanceo de comentarios entre texto escrito en inglés y texto escrito en otros idiomas, sabiendo que la cantidad de comentarios no es un problema, un modelo terminaría por aprender a clasificar sin fijarse en el lenguaje en el que está escrito.

4. Tuvimos problemas con la versión de NLTK porque aparentemente las versiones que manejábamos eran distintas y la funcionalidad cambiaba entre una y otra. Por favor, de encontrar algún problema en cuanto a la ejecución del código, nótese que puede ser debido a esto. Antes de suspendernos, consúltanos :‘(

5. La implementación de todos los códigos (salvo el de pruebas y casos de uso, *prueba.py*) de esta práctica es nuestra, si bien nos hemos ayudado en herramientas de inteligencia artificial generativa (Claude Sonnet 5.5) (sección 3), así como en los recursos bibliográficos propuestos en la práctica (sección 2, [1]).

4.2. Conclusiones

1. El corpus está desbalanceado hacia reseñas positivas. Esto es algo completamente irrelevante para esta primera práctica pero asumimos que no lo será para prácticas donde entrenemos modelos. Por ello, como hipótesis, creemos que la mejor forma de tratar esto sería el *undersampling* para ahorrar cómputo.

2. Las *stopwords* suponen una gran parte del texto, pero no apotan información especialmente relevante acerca de si la reseña es positiva, neutra o negativa (por eso se han eliminado todas a excepción de ’no’, ’nor’ y ’not’).

3.

Referencias

[1] B. Bengfort, R. Bilbro y T. Ojeda, *Applied Text Analysis with Python: Enabling Language-Aware Data Products with Machine Learning*, 1.ª ed., O'Reilly Media, Sebastopol, CA, 2018. ISBN 978-1-4919-6304-3.